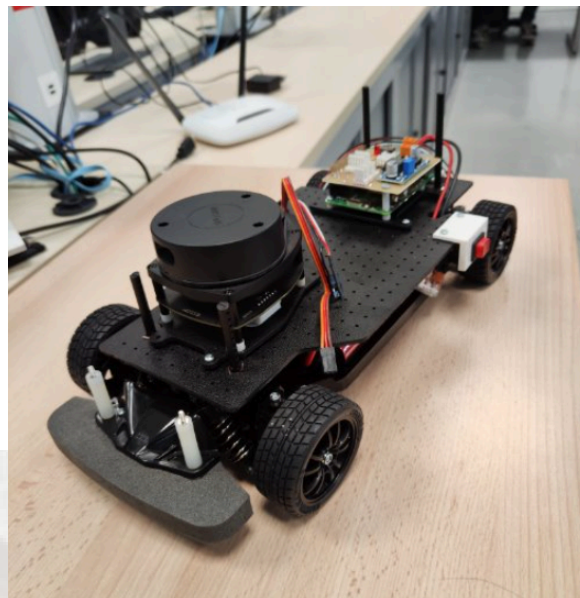


Objectif de ce projet :

Concevoir la partie logicielle
(POO/C++/Qt) d'une voiture
autonome



Sommaire Rapport Final Voiture Autonome :

I. Descriptif du cahier des charges

II. Explications des solutions

III. Bilan / Perspectives

IV. Bilan sur le travail de groupe

V. Diagramme du réseau IP

VI. Diagramme de classes

VII. Diagramme d'objets

Descriptif du cahier des charges

1. Objectifs généraux du projet

- Concevoir et programmer un véhicule autonome capable de suivre un circuit en simulation, puis d'évoluer dans un environnement réel en respectant des contraintes de navigation, de détection d'obstacles et de trajectoire.
- Développer les compétences en systèmes embarqués, en programmation (C++, Qt) et en gestion de capteurs (LiDAR, GPIO, PWM).

2. Fonctionnalités principales

- **Voiture simulée :**
 - Parcourir un circuit défini sans sortir des limites.
 - Gérer des circuits plus complexes, incluant des manœuvres comme la marche arrière.
 - Ajouter une interface déportée pour contrôler le véhicule et visualiser les informations du trajet.
- **Voiture réelle :**
 - Éviter les obstacles en temps réel.
 - Ajouter une interface déportée pour le contrôle et l'affichage des informations de navigation.
 - Effectuer des manœuvres, notamment la marche arrière, lorsque nécessaire pour continuer sur le parcours.
 - Suivre un parcours prédéfini, incluant la gestion des croisements de route.

3. Exigences techniques

- Utilisation de capteurs (LiDAR, capteurs de distance) pour la détection d'obstacles et la mesure de la distance parcourue.
- Programmation en C++ et utilisation de Qt Creator pour l'interface et la programmation du véhicule.
- Configuration des entrées/sorties (GPIO) et des signaux PWM pour la gestion des moteurs et des capteurs.

4. Contraintes de performance

- Fiabilité dans le suivi de trajectoire
- Optimisation de l'algorithme de parcours pour minimiser les erreurs et maximiser la fluidité de mouvement.
- Interface utilisateur intuitive et réactive pour la prise en main et le contrôle à distance.

5. Critères d'évaluation

- **Précision :** Respect des limites de circuit et réussite des manœuvres de parcours.
- **Efficacité :** Capacité du véhicule à suivre un parcours et à prendre les virages de manière fluide.
- **Qualité de l'interface :** Clarté, ergonomie et facilité de prise en main de l'interface de contrôle.
- **Documentation :** Rédaction d'un rapport final complet et d'une soutenance explicative, en français et en anglais.
- **Démonstration :** Réalisation d'une démonstration pratique des fonctionnalités de notre véhicule

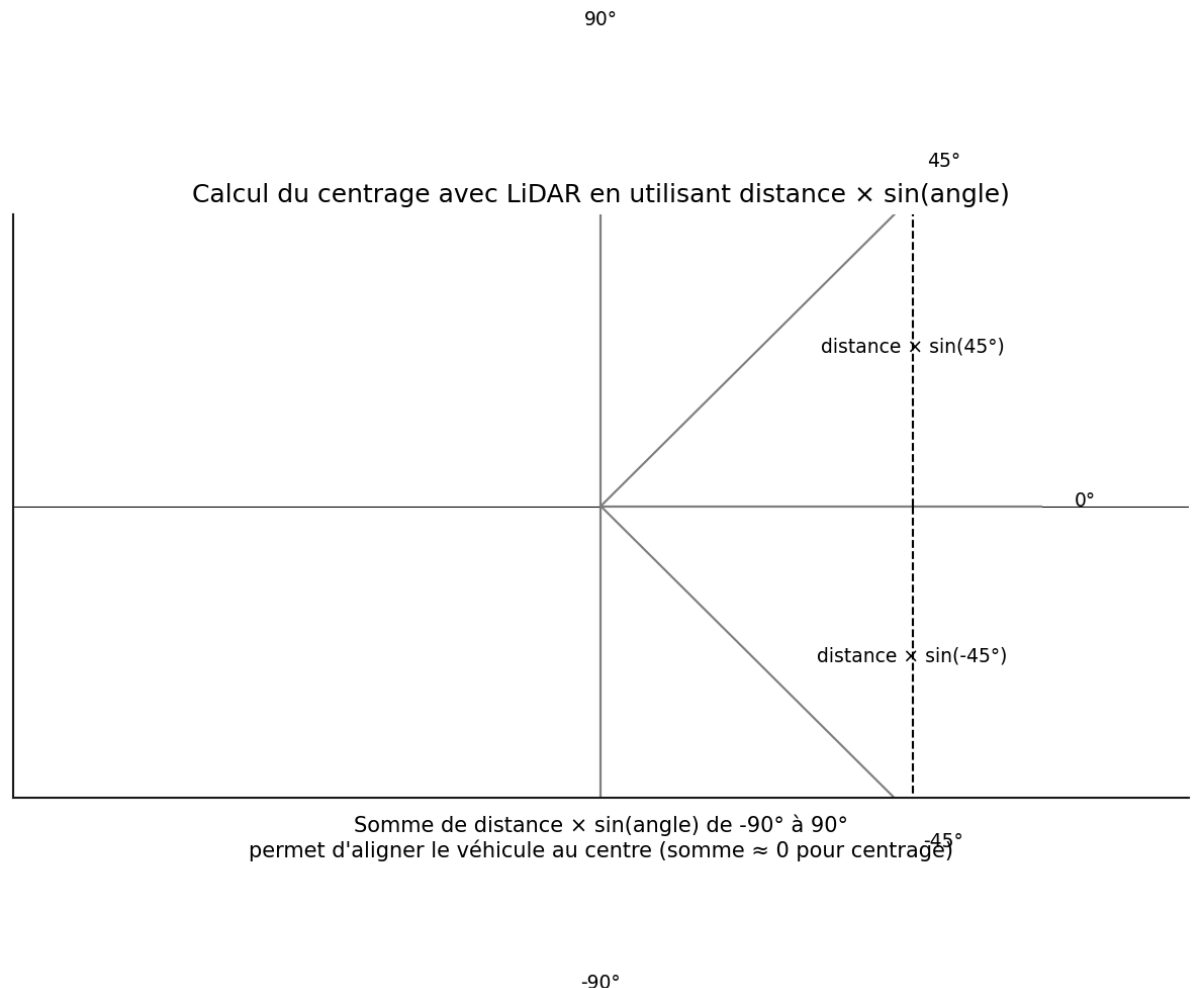
6. Livrables attendus

- **Code source :** Développement complet du code en C++ avec la documentation.
- **Interface graphique :** Interface Qt fonctionnelle pour la prise en main du véhicule.
- **Documentation :** Rapport détaillant les choix techniques, le code et les résultats de tests.
- **Démonstration finale :** Présentation pratique de la voiture autonome dans un environnement réel.

Explications des solutions

Pour la conception du véhicule autonome, plusieurs solutions techniques ont été explorées et implémentées, notamment :

1. Centrage entre les murs :



Le LiDAR nous fournit 360 valeurs de distance correspondant à des angles allant de -180° à 180° . Pour centrer le véhicule entre deux murs, nous utilisons les distances mesurées pour les angles entre **-90° et 90°** .

Pour chaque angle dans cette plage, nous calculons le produit de la distance mesurée par le **sinus de l'angle** :

$$\text{Valeur} = \text{distance} \times \sin(\text{angle})$$

Ce calcul nous donne le **côté opposé** du triangle formé par le rayon du LiDAR à cet angle, ce qui correspond à la distance latérale par rapport à l'axe central du véhicule.

En faisant la **somme de ces valeurs** pour tous les angles de -90° à 90° , nous obtenons une mesure globale de la position du véhicule par rapport au centre entre les deux murs. Si la somme est nulle, cela signifie que le véhicule est parfaitement centré. Une somme positive ou négative indique une déviation vers la gauche ou la droite.

Ainsi, en ajustant la direction du véhicule pour ramener cette somme à zéro, nous pouvons centrer le véhicule entre les murs.

2. Régulation de la vitesse :

Pour chaque angle dans cette plage, nous calculons le produit de la distance mesurée par le sinus de l'angle :

$$\text{Valeur} = \text{distance} \times \sin(\text{angle})$$

Ce calcul nous donne le côté opposé du triangle formé par le rayon du LiDAR à cet angle, indiquant la distance latérale entre le véhicule et le mur adjacent.

En faisant la somme de ces valeurs pour tous les angles de -90° à 90° , nous obtenons une mesure de la **proximité globale** des murs. Plus cette somme est faible, plus les murs sont proches. En conséquence, la vitesse du véhicule est réduite lorsque la somme est faible (mur proche), et légèrement augmentée lorsque la somme est plus élevée (mur plus éloigné), permettant une régulation de vitesse dynamique.

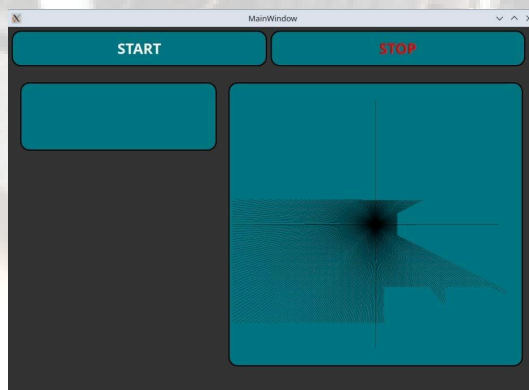
Ainsi, en ajustant la vitesse en fonction de cette somme, le véhicule peut se déplacer en toute sécurité, ralentissant lorsqu'il se rapproche des murs et accélérant lorsqu'il est bien centré.

3. Communication TCP entre les ordinateurs :

- L'ordinateur 2, agissant en tant que serveur TCP, envoie des données en temps réel à l'ordinateur 1 pour afficher les informations essentielles sur l'interface graphique, comme la distance aux murs et la position de la voiture.
- De plus, cet ordinateur contrôle la voiture en communiquant à la fois avec le simulateur ou la voiture réelle, permettant ainsi une gestion centralisée des actions du véhicule, même en présence de données en provenance de plusieurs sources.

4. Interface graphique déportée :

- L'interface graphique, créée avec Qt, permet de surveiller en temps réel les données de navigation, rendant l'interaction plus ergonomique et facilitant l'observation des comportements de la voiture sans intervention directe sur le véhicule. Elle contient deux boutons poussoirs START et STOP pour envoyer respectivement ON et OFF au serveur TCP qui contrôle la voiture nous avons ensuite un champ de texte où nous allons afficher la vitesse et l'angle de braquage de la voiture et finalement nous avons un widget qui transcrit en temps réel ce que voit notre capteur lidar 360 sur notre voiture



Bilan / Perspectives

1. Bilan sur l'adéquation par rapport au cahier des charges

- **Centrage entre les murs** : Le LiDAR 360 installé sur la voiture permet d'obtenir les distances par rapport aux murs de chaque côté. Cette donnée est utilisée pour calculer une moyenne, permettant ainsi à la voiture de rester centrée entre les deux murs. Cette fonctionnalité remplit l'objectif de suivi de trajectoire
- **Absence de marche arrière** : La fonctionnalité de marche arrière n'a pas été intégrée, faute de temps pour exploiter le capteur LiDAR arrière, ce qui limite les manœuvres dans les parcours complexes ou lors de blocages.

2. Manques par rapport au cahier des charges

- **Évitement d'obstacles** : Bien que le LiDAR 360 permette une prise en compte des murs pour le centrage, il n'est pas utilisé pour un évitement d'obstacles actif. Cette limitation empêche la voiture d'anticiper et de réagir aux obstacles de manière optimale.
- **Gestion des collisions** : L'absence d'algorithmes de suivi et de détection active limite la capacité du véhicule à interagir avec d'autres voitures sur le même circuit, ce qui restreint son autonomie dans des environnements partagés.

3. Améliorations potentielles

- **Ajout de la marche arrière via le capteur arrière** : Exploiter le LiDAR arrière pour sécuriser et automatiser la marche arrière permettrait des manœuvres plus complètes et fluides.
- **Optimisation du centrage et du suivi** : Améliorer l'algorithme de centrage pour une meilleure stabilité, et tirer parti des informations supplémentaires du LiDAR 360 pour un suivi de trajectoire plus précis et rapide en ajoutant un PID au lieu d'un simple proportionnelle.

4. Conclusion

Le projet bénéficie d'une base solide avec le LiDAR 360 pour le centrage entre deux murs. Cependant, l'ajout d'un système d'évitement actif et l'utilisation du capteur arrière pour la marche arrière permettraient d'étendre les capacités de navigation et de répondre à un cahier des charges plus avancé, notamment dans des environnements avec plusieurs véhicules et obstacles variables.

Bilan sur le travail de groupe

Le travail de groupe s'est déroulé de manière collaborative et a permis de répartir les tâches selon les compétences et intérêts de chaque membre :

1. Répartition des responsabilités : Chaque membre a pris en charge des parties spécifiques du projet, comme la configuration réseau, le développement de l'interface graphique, et la programmation de la logique de navigation. Cette division a permis de progresser efficacement dans les différentes phases du projet, tout en respectant les délais.

2. Communication et coordination : Des réunions régulières ont été organisées pour faire le point sur l'avancement, les difficultés rencontrées et les solutions possibles. Cela a permis d'aligner les objectifs de chaque membre et d'assurer une cohésion dans le développement des différentes parties du projet.

3. Adaptation face aux imprévus : Certaines fonctionnalités, comme la marche arrière et l'évitement actif des obstacles, n'ont pas pu être intégrées par manque de temps. Cependant, l'équipe a su adapter les priorités en fonction des contraintes rencontrées, en se concentrant sur les fonctionnalités principales et les exigences essentielles du cahier des charges.

4. Amélioration continue : Tout au long du projet, des ajustements ont été apportés pour optimiser les performances, notamment dans la gestion des données de centrage et dans l'interface utilisateur. Cette approche itérative a permis de peaufiner progressivement la solution, améliorant la stabilité et la précision du système.

Ce bilan met en évidence une collaboration productive, malgré les défis techniques et temporels, avec une répartition des tâches qui a permis d'atteindre les objectifs principaux du projet.

Diagramme du réseau IP

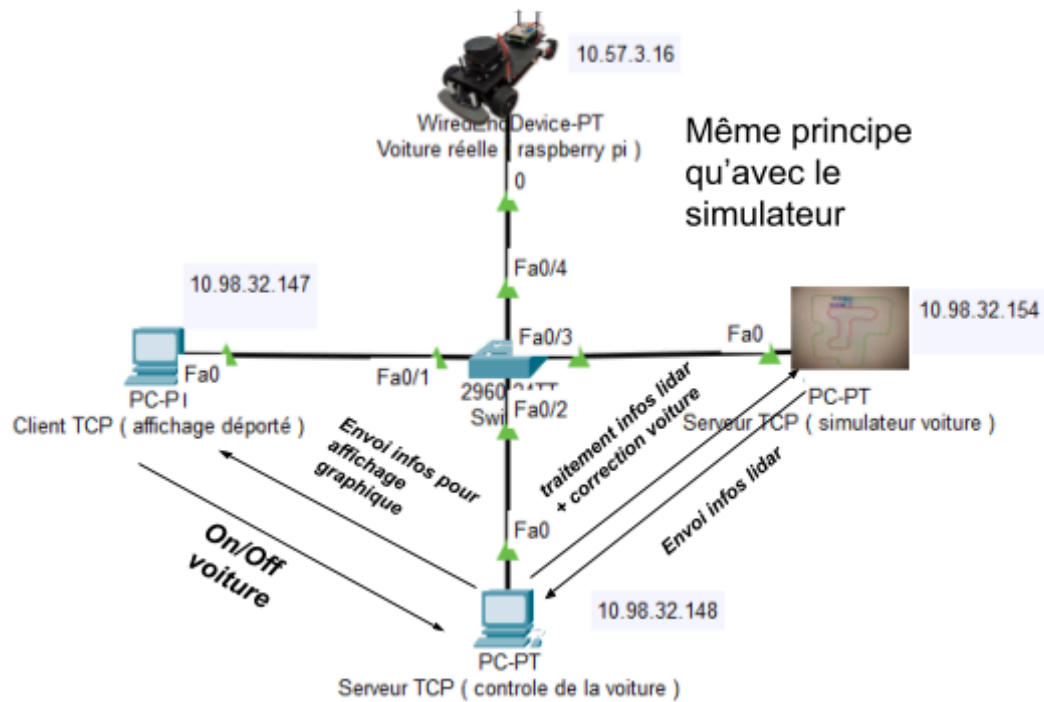
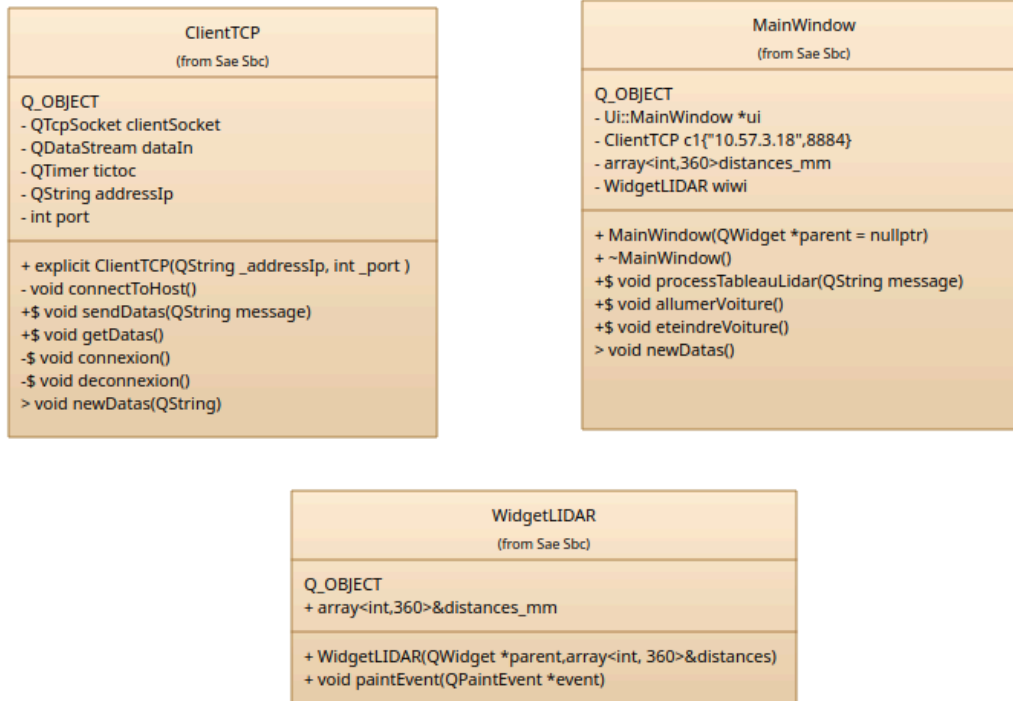


Diagramme de classes

Partie affichage déporté (Atamna) :



Partie affichage contrôle voiture (Samahri) :

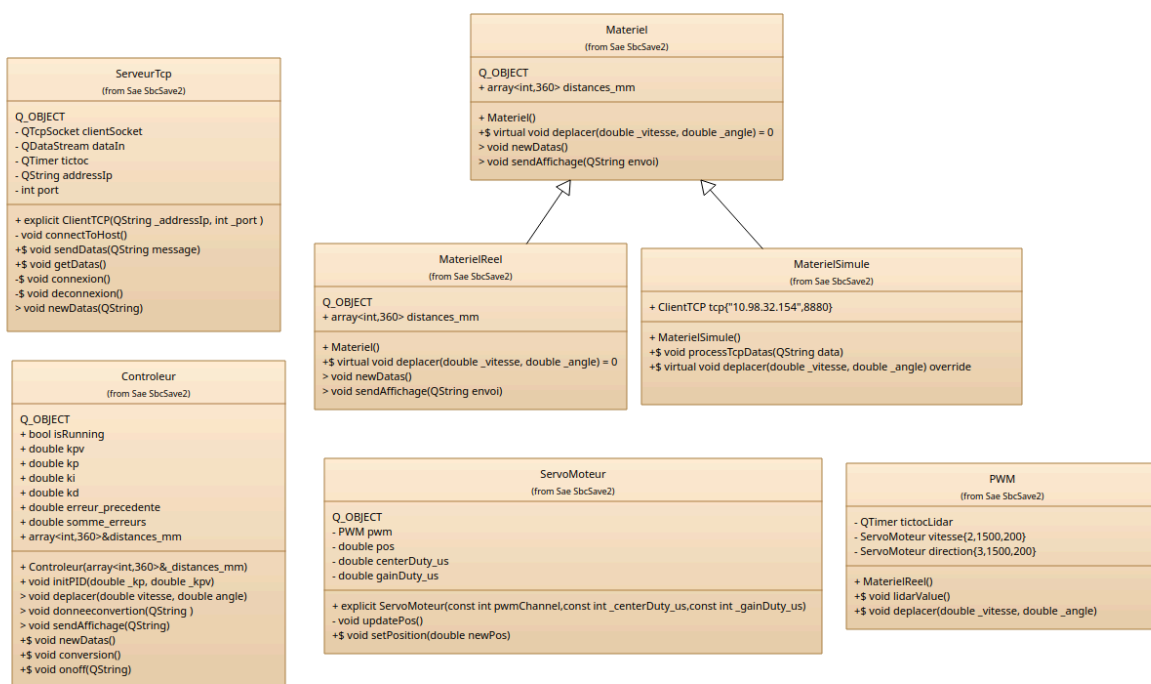
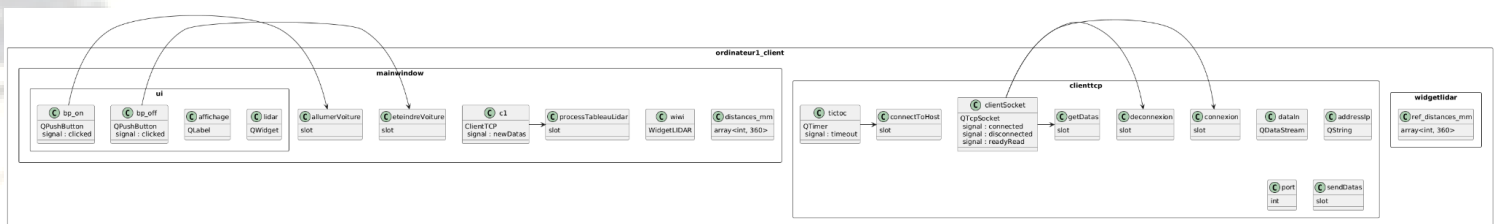
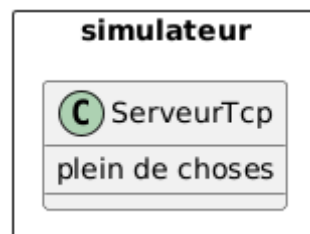


Diagramme d'objets

Partie affichage déporté (Atamna) : le symbole & pour le passage en référence n'était pas accepté d'où la mention "ref"



Partie simulateur que nous n'avons pas eu à traiter



Partie contrôle de la voiture (Samahri) :

